CS243X

THE UNIVERSITY OF WARWICK
Second Year Examinations, January 2014
Data Structures and Algorithms & Algorithm Design

- **Time allowed: 1.5 hours per SECTION (combined 3 hours).**
- Answer **TWO** questions **from each section** for which you are registered.
- Use **separate answerbooks** for answering questions from the two sections. In other words, put all of your answers to questions from section A into one answerbook, and put all of your answers to questions from section B into the other answerbook. Make it clear on the cover of each answerbook which section (A or B) it is for, and which two questions from the respective section you have selected.
- Calculators are not required and not permitted.

Continued

CS243X

Section A: Data Structures and Algorithms (CS 243)

- **Time allowed for this section: 1.5 hours.**
- Answer **TWO** questions from this section. Make it clear on the cover of the answerbook which two questions you have selected.
- Each question is worth a total of 50 marks. The nominal credit mark assigned to each part of a question is indicated by a bold figure next to that part.

Continued

CS243X

1. Imagine that you are preparing for a canoeing trip going from the source of a river to the sea. There are n villages along the river, numbered from n (at the source of the river) to 1 (on the sea). In every village you can rent a canoe which can be returned at any other village downstream. For every pair of villages i and j (such that $i > j$, i.e., such that j is downstream from i) we know the price $p[i, j]$ of renting a canoe in village i and returning it in village j.

   A *schedule* from n to 1 is a list of villages whose rental shops you should use on your way down the river from village n to village 1. For example, if $n = 4$ then $(4, 3, 1)$ is a schedule in which you rent a canoe from village 4 to village 3, and then you rent from village 3 to village 1.

   (a) **[12 marks]** Let $n = 4$ and consider the following array p of rental prices:

   p

	1	2	3	4
1				
2	5			
3	7	4		
4	8	2	3	

   that is, assume that the cost of renting a canoe from 3 to 2 is 4, the cost of renting a canoe from 4 to 1 is 8, etc.

   The total rental cost of a schedule is defined as the sum of all the individual rental prices; for example the total rental cost of the schedule $(4, 3, 1)$, for the above array of prices, is $p[4, 3] + p[3, 1] = 3 + 7 = 10$.

   i. How many schedules are there for a canoe trip from 4 to 1? List all of them and calculate their total rental costs.

   ii. What is the total rental cost of the *optimal* schedule, i.e., the schedule that minimises the total rental cost? Which schedule is optimal?

   (b) **[12 marks]** Suppose that the total costs $OC[j-1], OC[j-2], \ldots, OC[2], OC[1]$ of optimal schedules from villages $j-1$, $j-2$, ..., 2, and 1, respectively, to village 1, have already been computed. Characterise the total cost $OC[j]$ of an optimal schedule from j to 1, using the values $OC[j-1], OC[j-2], \ldots, OC[2], OC[1]$.

   (c) **[10 marks]** Design a polynomial-time algorithm that—given an array $p[i, j]$ of prices, for $n \geq i > j \geq 1$, as input—computes the total rental cost $OC[n]$ of an optimal schedule from n to 1. What is the running time of your algorithm as a function of n?

   (d) **[8 marks]** Explain how you can modify your algorithm so that it outputs not only the total cost of an optimal schedule from n to 1, but also an optimal schedule itself.

   (e) **[8 marks]** Illustrate the behaviour of your algorithm on the example from part (a), i.e., show the steps which it takes to compute the total cost of an optimal schedule (and an optimal schedule itself).

Continued

2. Consider a university course that consists of n modules called 1, 2, 3, ..., n. Every module has a list of prerequisites, which are other modules. The *prerequisites graph* has vertices 1, 2, 3, ..., n, and it has a directed edge from j to i if module j occurs on the list of prerequisites of module i. (For simplicity, we assume that if a module j is a prerequisite for module i then $j < i$; note that this implies that there are no cycles in the prerequisites graph.)

   (a) **[8 marks]** Consider a course with modules 1, 2, 3, 4, 5, and 6, and with the following lists of prerequisites:

Module	Prerequisites
1	
2	1
3	1
4	3
5	
6	2, 4, 5

   Draw the prerequisites graph of this course.

   (b) **[12 marks]** A course timetable T is an assignment of modules to terms (terms are numbered 1, 2, 3, ...). For example, $T[3] = 2$ means that module 3 is taught in term 2. We say that a course timetable T is correct if for every pair of modules i and j, such that j is on the prerequisites list of i, we have that $T[j] < T[i]$ (i.e., module j is taught before module i).

   The following timetable:

Module i	1	2	3	4	5	6
Term $T[i]$	1	3	2	4	3	5

   is correct for the example from part (a) and it uses five terms for the whole course.

   Provide another correct timetable for the course from part (a) that uses only four terms. Is there a timetable for this course that uses less than four terms? If not, then argue why. If yes, then give one.

   (c) **[16 marks]** Design an efficient polynomial-time algorithm that—given the lists of prerequisites for all modules—computes the smallest number of terms sufficient to construct a correct course timetable.

   What is the running time of your algorithm? Full credit will only be given if your algorithm runs in time linear in the size of the prerequisites graph.

   (d) **[8 marks]** An *optimal timetable* is a correct timetable that uses the smallest possible number of terms. Discuss how you can modify your algorithm from part (d) so that it outputs an optimal timetable.

   (e) **[6 marks]** Illustrate the behaviour of your algorithm for the example list of modules and their prerequisites from part (a).

Continued

CS243X

3. We have to schedule n jobs, one after another, on one machine. The *processing times* of jobs $1, 2, \ldots, n$ are positive numbers $t[1], t[2], \ldots, t[n]$, and their *urgency factors* are positive numbers $u[1], u[2], \ldots, u[n]$, respectively.

   The *completion time* of a job in a schedule $S = (s_1, s_2, \ldots, s_n)$ is the time it waits until it has been processed; in other words, the completion time of job s_i in schedule S is:

   $$c_S(s_i) = t[s_1] + t[s_2] + \cdots + t[s_i].$$

   The *lateness* (urgency-weighted completion time) of job j in schedule S is defined to be:

   $$\ell_S(j) = u[j] \cdot c_S(j).$$

   The *total lateness* of schedule S is defined by:

   $$L_S = \ell_S(1) + \ell_S(2) + \cdots + \ell_S(n).$$

   We are interested in finding a schedule that minimises total lateness.

   (a) **[12 marks]** Suppose that we have three jobs $1, 2$, and 3 with the following processing times and urgency factors:

Job j	1	2	3
Processing time $t[j]$	5	2	3
Urgency factor $u[j]$	3	1	2

   Consider the schedule $S = (1, 3, 2)$ that first completes job 1, then job 3, and finally job 2.

   What are the completion times $c_S(1), c_S(2)$, and $c_S(3)$ of jobs 1, 2, and 3 in schedule S?

   What is the lateness $\ell_S(1), \ell_S(2)$, and $\ell_S(3)$ of jobs 1, 2, and 3 in schedule S?

   What is the total lateness L_S of schedule S?

   (b) **[10 marks]** Note that if there are just two jobs 1 and 2, then there are just two possible schedules: $(1, 2)$ and $(2, 1)$.

   Write the expression $L_{(1,2)} - L_{(2,1)}$ as a function of $t[1]$, $t[2]$, $u[1]$, and $u[2]$, and analyse its sign to characterise when schedule $(1, 2)$ has smaller total lateness than schedule $(2, 1)$.

   (c) **[20 marks]** Design a polynomial-time algorithm that—given the tables $t[1..n]$ and $u[1..n]$ of processing times and urgency factors of jobs $1, 2, \ldots, n$—computes the schedule that minimises total lateness.

   (d) **[8 marks]** For the input in part (a), what is the schedule that minimises total lateness? What is the total lateness of this schedule?

CS243X

Section B: Algorithm Design (CS 244)

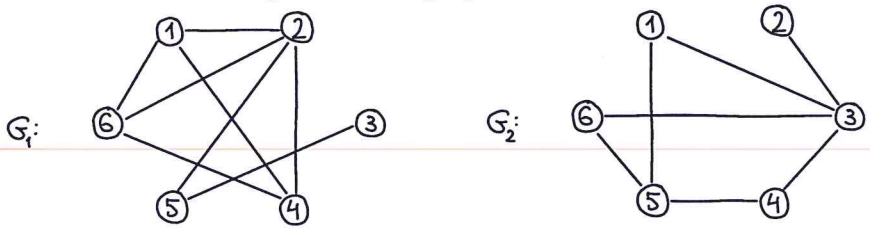
- **Time allowed for this section: 1.5 hours.**
- Answer **TWO** questions from this section. Make it clear on the cover of the answerbook which two questions you have selected.
- Each question is worth a total of 50 marks. The nominal credit mark assigned to each part of a question is indicated by a bold figure next to that part.

Continued

CS243X

1. (a) **[8 marks]** A *clique* in an undirected graph $G = (V, E)$ is a subset $C \subseteq V$ of vertices in which every pair of vertices is connected by an edge. (In other words, $C \subseteq V$ is a clique if for all $v, w \in C$, we have $\{v, w\} \in E$.) The size of a clique C is defined to be the number of vertices in set C.

   Consider the following undirected graphs G_1 and G_2.



   i. What is the maximum size of a clique in graph G_1? Give a clique of maximum size in G_1.

   ii. What is the maximum size of an independent set in graph G_2? Give an independent set of maximum size in G_2.

   (b) **[15 marks]** The CLIQUE decision problem is—given a graph G and a number k—to determine if there is a clique of size k in G. Prove that the CLIQUE problem is NP-complete.

   You may assume that the INDEPENDENT-SET problem is NP-complete. The INDEPENDENT-SET decision problem is—given a graph G and a number k—to determine if there is an independent set of size k in G.

   (c) **[15 marks]** The DENSE-SUBGRAPH decision problem is—given a graph G and numbers k and d—to determine if there is a set of k vertices of G such that there are at least d edges between them. Prove that the DENSE-SUBGRAPH problem is NP-complete.

   (d) **[12 marks]** A path in a graph is simple if it does not visit any vertex more than once.

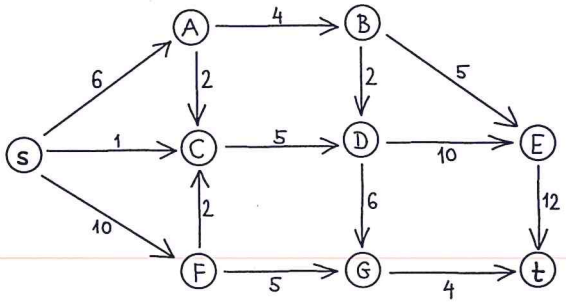
   The HAMILTONIAN-PATH decision problem is—given a directed graph G, and given two vertices s and t in G—to determine if there is a simple path from s to t that visits every vertex in graph G.

   The FIND-HAMILTONIAN-PATH *search problem* is—given a directed graph G, and given two vertices s and t in G—to output a simple path from s to t that visits every vertex in graph G, if there is one.

   Prove that there is a polynomial-time reduction from the FIND-HAMILTONIAN-PATH search problem to the HAMILTONIAN-PATH decision problem. In other words, give a polynomial-time algorithm for computing "Hamiltonian paths", which can use an oracle that determines if a Hamiltonian path exists in a given graph.

Continued

2. (a) **[15 marks]** Consider the following flow network. The numbers labelling the edges are their capacities, and s and t are the distinguished *source* and *sink* vertices.



Give a maximum-value flow in the network. Prove that it is indeed a maximum-value flow by exhibiting an s-t-cut in the network whose capacity is equal to the value of the flow.

(b) **[15 marks]** For the flow f from part (a), draw the residual graph G_f, including residual capacities of all edges.

Give the set of vertices in the residual graph that can be reached from the source vertex, and the set of vertices from which the sink can be reached.

(c) **[10 marks]** An edge in a network is called a *bottleneck edge* if increasing its capacity (while the capacities of all other edges stay the same) results in an increase of the value of a maximum-value flow. List all bottleneck edges in the above network and justify your answer.

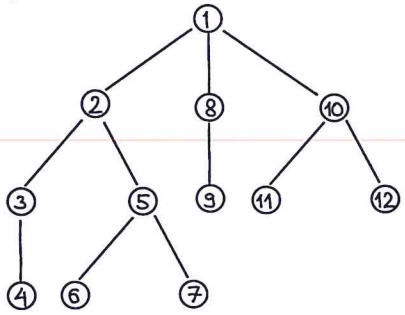
(d) **[10 marks]** Let G be a flow network with integer capacities. Describe a polynomial-time algorithm that—given such a flow network—outputs all of its bottleneck edges.

Continued

CS243X

3. A *matching* in an undirected graph $G = (V, E)$ is a subset $M \subseteq E$ of edges in which no two edges share a vertex. The size of a matching M is the number of edges in set M.

   Consider undirected graphs that are undirected rooted trees, represented by a parent array p, such that if a vertex v is the root then $p[v] = v$, and otherwise $p[v]$ is the parent of v. For example, the following undirected rooted tree G:



   is represented by the parent array:

Vertex v	1	2	3	4	5	6	7	8	9	10	11	12
Parent $p[v]$ of v	1	1	2	3	2	5	5	1	8	1	10	10

   (a) **[6 marks]** Are sets $\{\{2,5\},\{8,9\},\{10,12\}\}$ and $\{\{10,12\},\{5,7\},\{1,10\}\}$ matchings in the example tree G above? Explain your answers. Give a matching in G of size 4.

   (b) **[6 marks]** What is the maximum size of a matching in the example tree G above? Give a matching of G of maximum size.

   (c) **[6 marks]** Prove that every undirected tree is a bipartite graph.

   (d) **[14 marks]** Design a polynomial-time algorithm that—given an undirected rooted tree as input—computes the maximum size of a matching in the tree.

   Justify the correctness of your algorithm. In order to achieve maximum credit, your algorithm should have linear running time; generous credit will be given for slower algorithms.

   (e) **[8 marks]** Draw the tree G with weighted vertices represented by the following parent array:

Vertex v	1	2	3	4	5	6	7	8	9
Parent $p[v]$ of v	1	1	2	2	1	5	1	7	7
Weight $w[v]$ of v	3	4	1	2	1	2	5	1	1

   Give an independent set in G of maximum size. Give an independent set in G that has maximum total weight.

   (f) **[10 marks]** Design a polynomial-time algorithm that—given a tree with weighted vertices as input—computes the maximum total weight of an independent set in the tree.